

- Q1: Explain Bubble Sort algorithm in detail. | Marks: 5
 - Model Answer: Bubble Sort repeatedly swaps adjacent elements if they are in the wrong order. It performs multiple passes; after each pass, the largest among unsorted items moves to its correct position. Time complexity is $O(n^2)$ in average/worst case; best case $O(n)$ if optimized with a swapped flag. It is stable and works in-place.
- Q2a: Define a Stack and its basic operations. | Marks: 2
 - Model Answer: A Stack is a LIFO (Last-In-First-Out) data structure. Core operations: [push](#) to add an element to the top, pop to remove the top element, and peek to inspect the top without removal.
- Q2b: Explain push and pop operations with an example. | Marks: 3
 - Model Answer: [push\(x\)](#) places x at the top; pop() removes and returns the top element. Example: Starting with empty stack, push(5), push(7) → top is 7. pop() returns 7, stack now [5].
- Q3: Differentiate between Queue and Stack. | Marks: 4
 - Model Answer: Stack uses LIFO; Queue uses FIFO (First-In-First-Out). In a Stack, the last inserted is first removed; in a Queue, the first inserted is first removed. Queues support enqueue and dequeue. Typical use: recursion stacks, browser history vs. scheduling queues.
- Q4: What is Big-O notation? Give two examples. | Marks: 3
 - Model Answer: Big-O expresses upper bound of growth rate relative to input size. Examples: $O(n)$ for linear traversal; $O(n \log n)$ for efficient sorting like mergesort/quick sort (average).
- Q5: Write the time complexity of binary search and explain why. | Marks: 3
 - Model Answer: Binary search is $O(\log n)$ because it halves the search space each iteration by comparing the middle element and discarding half the array, continuing until found or empty.